DOCUMENTACION

PROYECTO FINAL

Plantel: Unitec Sur

MATERIA: GESTION DE BASE DE DATOS
Arreola Muñoz José Eduardo
Romano Rodríguez Victor Ángel
- Docente: Miguel Ángel Aguilar Cruz

INDICE

1. Introducción 2

2. Objetivo 3

Objetivo general 3

Objetivos específicos 3

3. Arquitectura del Sistema 4

Capa de presentación (Frontend) 4

Capa de lógica de negocio (Backend) 4

Capa de datos (Base de datos) 4

Comunicación entre capas 5

Ventajas de la arquitectura 5

4.Requerimientos funcionales 6

5.-Requerimientos No Funcionales 8

6.-UML - DIAGRAMA DE CLASES 10

7. UML - Diagrama de Comunicación 11

Administrador 11

Instructor 12

Estudiante 13

Flujo general del sistema 13

8. UML – DIAGRAMA DE CASOS DE USO 14

Administrador 14

Docente 14

Estudiante 15

9. Seguridad 16

10. Auditoría 16

11. Lógica de Negocio 17

11. Conclusión 17

1. Introducción

En la actualidad, el uso de tecnologías web se ha convertido en un pilar fundamental para la gestión de información en instituciones educativas, permitiendo automatizar procesos, mejorar la accesibilidad y optimizar la administración de recursos. La digitalización de servicios académicos no solo facilita la interacción entre usuarios, sino que también garantiza un manejo más eficiente, seguro y centralizado de los datos.

El presente proyecto consiste en el desarrollo de un **Sistema de Gestión Universitaria** implementado bajo un enfoque de arquitectura distribuida, integrando tecnologías de desarrollo web y bases de datos. La solución está compuesta por una interfaz de usuario desarrollada con **HTML, CSS y Java**, que permite la interacción de los distintos tipos de usuarios del sistema (administradores, docentes y estudiantes), y un backend desarrollado en **Python**, encargado de procesar la lógica de negocio y gestionar la comunicación con la base de datos en **SQL Server**.

El sistema tiene como objetivo principal administrar procesos clave dentro de una institución educativa, tales como el registro de estudiantes, la gestión de cursos, la inscripción a materias, el registro de evaluaciones y la administración de pagos. Para lograrlo, se implementan mecanismos que garantizan la integridad, seguridad y consistencia de la información.

Uno de los aspectos más relevantes del sistema es la implementación de técnicas de seguridad a nivel de base de datos, incluyendo la encriptación de datos sensibles como correos electrónicos, documentos de identidad, comentarios y referencias de pago. Asimismo, se incorporan mecanismos de auditoría mediante triggers que permiten registrar automáticamente todas las operaciones críticas realizadas sobre la información, proporcionando trazabilidad y control.

Además, el sistema hace uso de procedimientos almacenados para encapsular la lógica de negocio, mejorando el rendimiento y asegurando la consistencia en las operaciones. Destaca especialmente la implementación de procesos transaccionales que integran múltiples operaciones en una sola ejecución, garantizando el cumplimiento de las propiedades ACID.

En conjunto, este sistema representa una solución integral que combina tecnologías web modernas con herramientas robustas de gestión de datos, proporcionando una plataforma eficiente, segura y escalable para la administración de procesos universitarios.

2. Objetivo

Objetivo general

Desarrollar un sistema integral que permita gestionar procesos académicos y administrativos de una universidad.

Objetivos específicos

- Registrar estudiantes e instructores
- Gestionar cursos y su disponibilidad
- Controlar inscripciones
- Registrar evaluaciones académicas
- Administrar pagos
- Garantizar seguridad y trazabilidad de datos

3. Arquitectura del Sistema

El sistema está diseñado bajo una arquitectura de tipo **cliente-servidor multicapa**, en la cual se separan claramente las responsabilidades entre la interfaz de usuario, la lógica de negocio y la gestión de datos. Esta estructura permite una mayor escalabilidad, mantenibilidad y seguridad.

Capa de presentación (Frontend)

Desarrollada utilizando **HTML, CSS y Java**, esta capa es la encargada de la interacción directa con el usuario. Proporciona interfaces gráficas para:

- Registro y consulta de estudiantes
- Inscripción a cursos
- Visualización de evaluaciones
- Gestión de pagos

Los distintos tipos de usuario (administrador, docente y estudiante) acceden a funcionalidades específicas según sus permisos.

Capa de lógica de negocio (Backend)

Implementada en **Python**, esta capa actúa como intermediaria entre la interfaz y la base de datos. Sus funciones principales incluyen:

- Procesamiento de solicitudes del usuario
- Validación de datos
- Ejecución de procedimientos almacenados
- Control de flujo de operaciones
- Manejo de errores y transacciones

Capa de datos (Base de datos)

Basada en **SQL Server**, esta capa gestiona el almacenamiento y procesamiento de la información. Incluye:

- Tablas relacionales
- Llaves primarias y foráneas
- Procedimientos almacenados
- Triggers para auditoría
- Mecanismos de encriptación

Comunicación entre capas

La comunicación entre el backend en Python y SQL Server se realiza mediante conectores especializados (como pyodbc), permitiendo ejecutar consultas, procedimientos y transacciones de forma eficiente.

Ventajas de la arquitectura

- Separación de responsabilidades
- Mayor seguridad en el manejo de datos
- Facilidad de mantenimiento
- Escalabilidad del sistema
- Posibilidad de migración a entornos web más complejos

En conjunto, esta arquitectura permite desarrollar un sistema robusto, flexible y alineado con prácticas modernas de desarrollo de software.

4.Requerimientos funcionales

ID	REQUERIMIENTO	DESCRIPCION	USUARIO	PRIORIDAD
RF-01	Registro de estudiantes	El sistema debe permitir registrar estudiantes con datos personales encriptados	Administrador	Alta
RF-02	Gestión de cursos	Permitir crear, editar y consultar cursos	Administrador	Alta
RF-03	Asignación de instructores	Permitir asignar uno o varios instructores a cursos	Administrador	Media
RF-04	Inscripción a cursos	Permitir inscribir estudiantes en cursos disponibles	Estudiante / Admin	Alta
RF-05	Control de cupo	Validar que no se exceda el cupo máximo del curso	Sistema	Alta
RF-06	Registro de evaluaciones	Permitir a los docentes registrar calificaciones y comentarios	Docente	Alta
RF-07	Consulta de evaluaciones	Permitir a estudiantes consultar sus evaluaciones	Estudiante	Media
RF-08	Registro de pagos	Permitir registrar pagos asociados a una inscripción	Administrador	Alta
RF-09	Gestión de métodos de pago	Permitir administrar métodos de pago disponibles	Administrador	Media
RF-10	Generación de reporte de demanda	Mostrar cursos con mayor número de inscripciones	Administrador	Media

RF-11	Proceso completo automatizado	Ejecutar inscripción, evaluación y pago en una sola transacción	Sistema	Alta
RF-12	Auditoría de operaciones	Registrar automáticamente inserciones, actualizaciones y eliminaciones	Sistema	Alta
RF-13	Consulta de información	Permitir consultar estudiantes, cursos e inscripciones	Todos	Alta
RF-14	Cancelación de inscripción	Permitir cambiar el estado de una inscripción	Administrador	Media

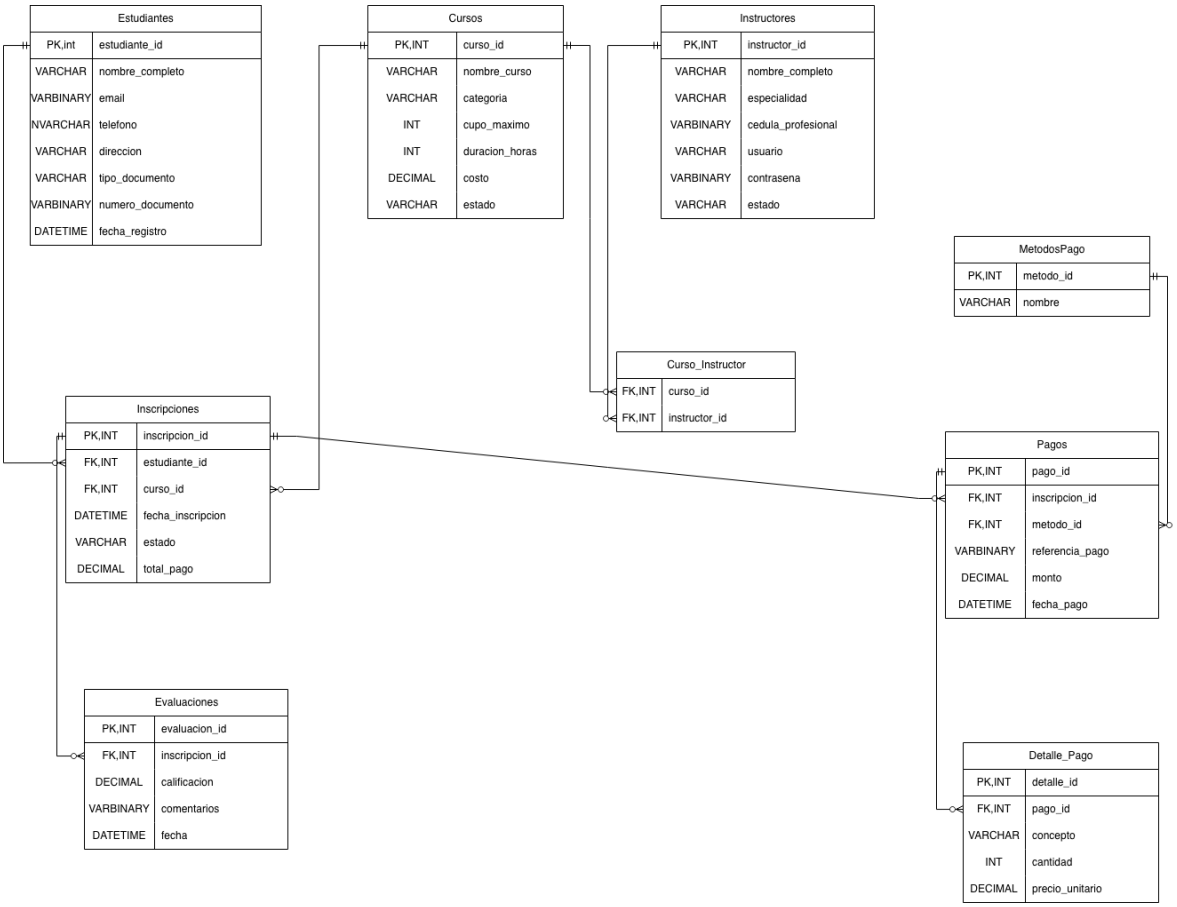
5.-Requerimientos No Funcionales

ID	REQUERIMIENTO	DESCRIPCION	TIPO	PRIORIDAD
RF-01	Seguridad de datos	Los datos sensibles deben almacenarse encriptados	Seguridad	Alta
RF-02	Integridad de datos	Uso de llaves primarias y foráneas para mantener consistencia	Base de datos	Alta
RF-03	Control de transacciones	Las operaciones críticas deben usar transacciones (ACID)	Rendimiento	Alta
RF-04	Auditoría	El sistema debe registrar cambios mediante triggers	Seguridad	Alta
RF-05	Disponibilidad	El sistema debe estar disponible durante el uso académico	Operativo	Media
RF-06	Escalabilidad	El sistema debe permitir agregar más cursos y usuarios	Arquitectura	Media
RF-07	Rendimiento	Las consultas deben ejecutarse eficientemente	Rendimiento	Media
RF-08	Usabilidad	La interfaz debe ser clara y fácil de usar	UX	Media
RF-09	Mantenibilidad	El sistema debe permitir modificaciones futuras fácilmente	Desarrollo	Media

RF-10	Compatibilidad	El sistema debe funcionar con SQL Server y Python	Técnico	Alta

6.-UML - DIAGRAMA DE CLASES

El siguiente diagrama representa la estructura lógica del sistema:



Este diagrama muestra:

- Las clases principales derivadas de la base de datos
- Sus atributos (campos)
- Relaciones entre clases mediante asociaciones

Cada clase corresponde directamente a una tabla en SQL Server, lo que permite una clara trazabilidad entre el diseño conceptual y la implementación física.

Las relaciones principales incluyen:

- Estudiante → Inscripciones (1)
- Curso → Inscripciones (1)
- Curso ↔ Instructor (N)
- Inscripción → Pago (1:1)
- Pago → DetallePago (1)

7. UML - Diagrama de Comunicación

El diagrama de comunicación representa la interacción entre los diferentes tipos de usuarios del sistema (Administrador, Instructor y Estudiante) y los módulos principales de la aplicación, mostrando el flujo de mensajes y operaciones que se realizan dentro del sistema.

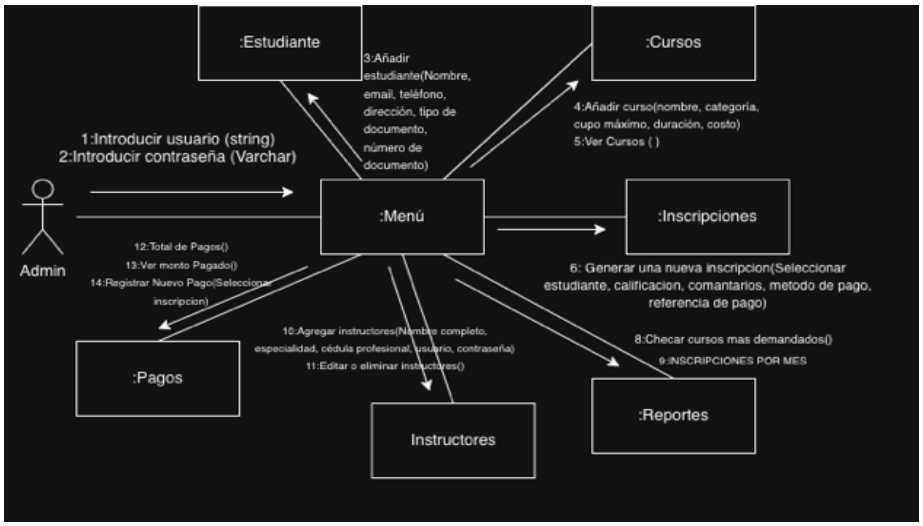
A diferencia de otros diagramas, este se enfoca en cómo se comunican los componentes del sistema a través de acciones específicas.

Administrador

El administrador interactúa con el sistema a través de un menú principal, desde donde puede gestionar todas las funcionalidades:

- Iniciar sesión mediante credenciales
- Registrar estudiantes (nombre, email, teléfono, dirección, documentos)
- Crear y consultar cursos
- Generar inscripciones completas (incluyendo evaluación y pago)
- Administrar instructores (agregar, editar y eliminar)
- Gestionar pagos (registro y consulta)
- Generar reportes:
    - Cursos más demandados
    - Inscripciones por mes

El administrador tiene control total del sistema, siendo el usuario con mayor nivel de acceso.

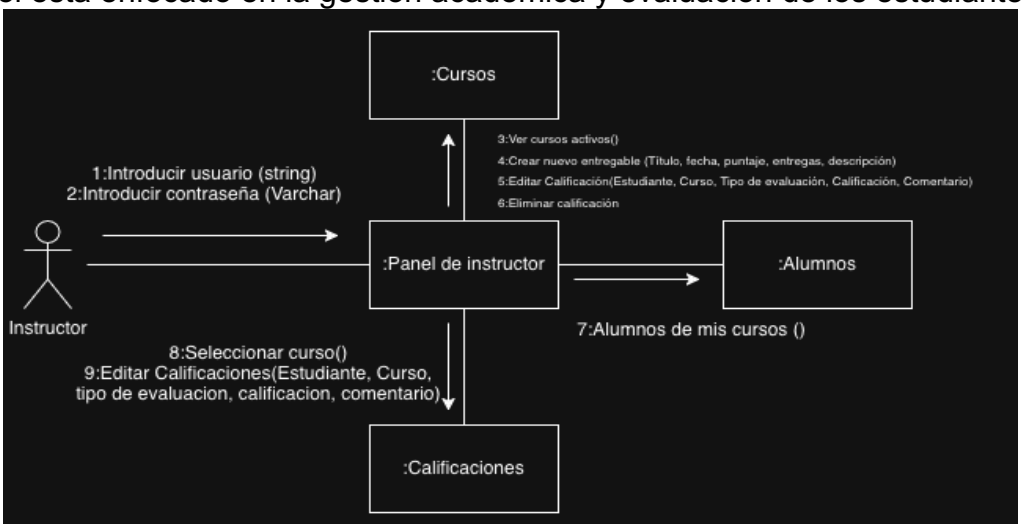


Instructor

El instructor accede al sistema mediante un panel específico que le permite interactuar con los cursos asignados:

- Iniciar sesión
- Visualizar cursos activos
- Consultar alumnos inscritos
- Seleccionar cursos específicos
- Registrar y editar calificaciones
- Crear evaluaciones
- Eliminar calificaciones si es necesario

Este rol está enfocado en la gestión académica y evaluación de los estudiantes.

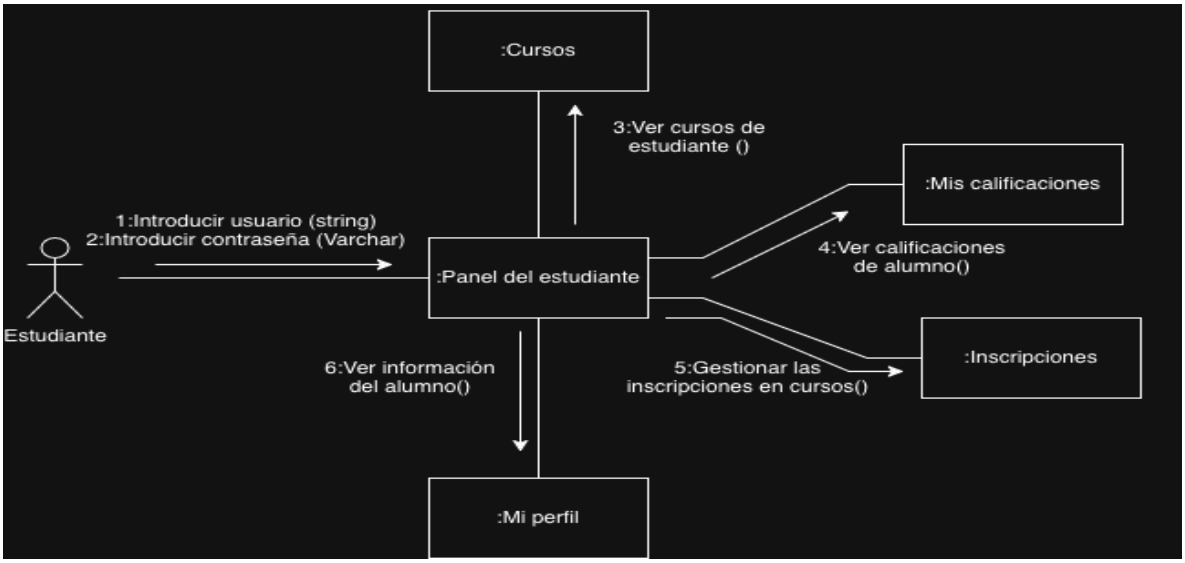


Estudiante

El estudiante utiliza un panel personal desde el cual puede gestionar su información y actividades académicas:

- Iniciar sesión
- Consultar cursos inscritos
- Gestionar inscripciones
- Visualizar sus calificaciones
- Consultar su información personal
- Acceder a su perfil

Este rol está orientado a la consulta y autogestión dentro del sistema.



Flujo general del sistema

El flujo de comunicación sigue el siguiente patrón:

1. El usuario ingresa sus credenciales en la interfaz (Frontend)
2. La solicitud es enviada al backend (Python)
3. Se valida la información y se ejecutan las operaciones necesarias
4. Se realizan consultas o modificaciones en la base de datos (SQL Server)
5. El sistema devuelve la respuesta al usuario

8. UML – DIAGRAMA DE CASOS DE USO

El diagrama de casos de uso representa las funcionalidades principales del sistema y la interacción de los diferentes tipos de usuarios con la plataforma. En este sistema se identifican tres actores principales: Administrador, Instructor y Estudiante.

Cada actor interactúa con el sistema a través de diferentes casos de uso, los cuales representan acciones específicas que pueden realizar dentro de la aplicación.

Administrador

El administrador tiene acceso a las funciones principales del sistema, incluyendo:

- Iniciar sesión
- Añadir estudiantes
- Añadir cursos
- Crear inscripciones
- Registrar pagos
- Gestionar instructores

Estas funcionalidades permiten la administración completa del sistema académico y financiero.

Docente

El docente interactúa con el sistema mediante funciones relacionadas con la gestión académica:

- Iniciar sesión
- Consultar cursos activos
- Visualizar alumnos inscritos
- Calificar alumnos

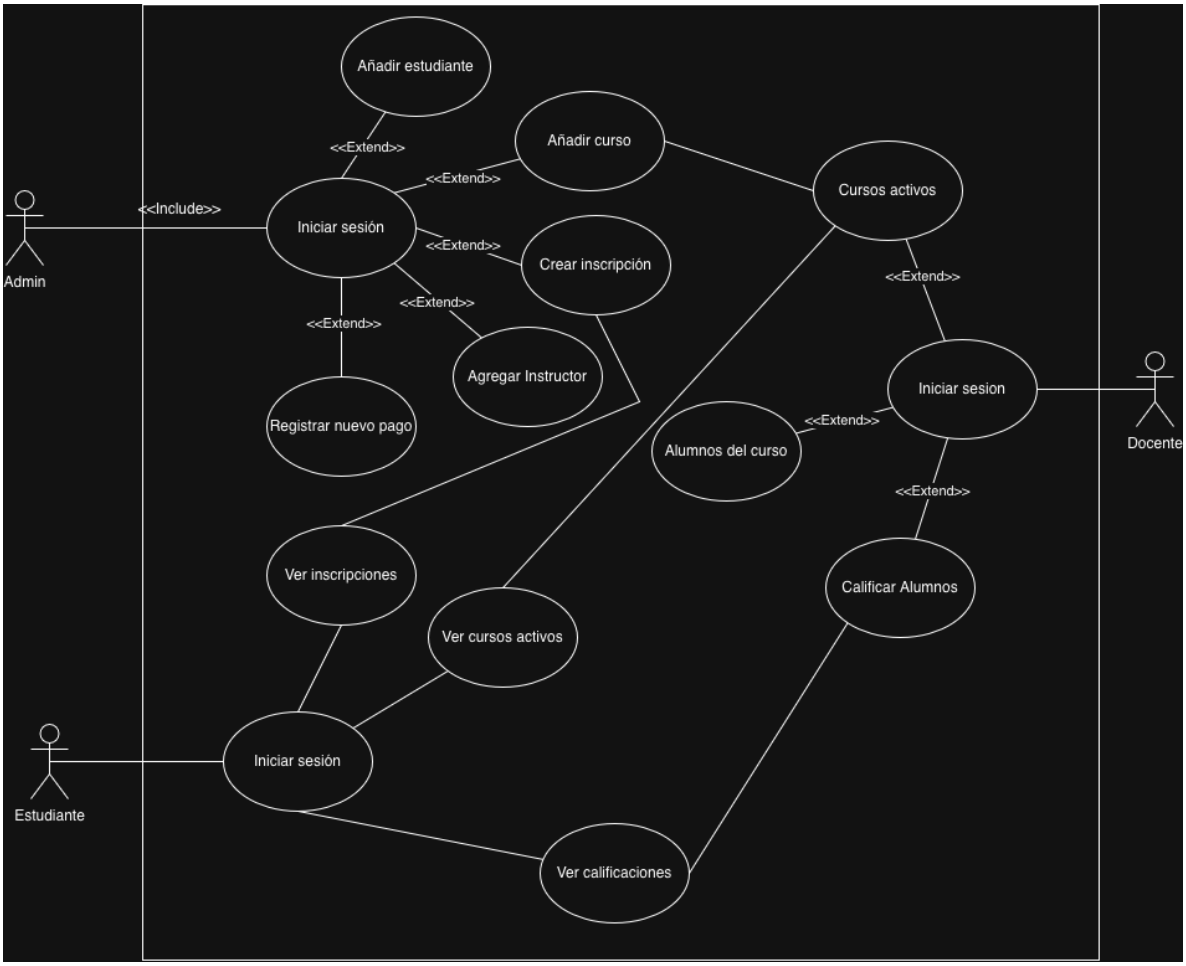
Estas acciones permiten al docente gestionar el desempeño académico de los estudiantes.

Estudiante

El estudiante accede al sistema para consultar su información académica:

- Iniciar sesión
- Ver cursos activos
- Consultar inscripciones
- Visualizar calificaciones

Este rol está enfocado en la consulta y seguimiento del progreso académico.



9. Seguridad

El sistema implementa:

- Encriptación de datos sensibles
- Protección de información crítica
- Control de acceso por tipo de usuario

10. Auditoría

Se implementan triggers para registrar:

- Inserciones
- Actualizaciones
- Eliminaciones

Esto permite mantener trazabilidad completa de los cambios en el sistema.

11. Lógica de Negocio

El sistema utiliza:

- Procedimientos almacenados
- Validaciones
- Control de cupo
- Transacciones

Destaca el procedimiento:
ProcesoEntero → integra inscripción, evaluación y pago en una sola operación.

11. Conclusión

El Sistema de Gestión Universitaria desarrollado integra de manera eficiente tecnologías web y de bases de datos bajo una arquitectura multicapa, permitiendo una clara separación de responsabilidades y facilitando su mantenimiento y escalabilidad.

A través del uso de Python, SQL Server y tecnologías de frontend, se logró implementar una solución capaz de gestionar procesos clave como inscripciones, evaluaciones y pagos, garantizando la integridad y consistencia de los datos mediante el uso de transacciones y procedimientos almacenados.

Asimismo, la incorporación de mecanismos de seguridad y auditoría, como la encriptación y los triggers, fortalece la protección y trazabilidad de la información.

En conjunto, el sistema cumple con los objetivos planteados y constituye una solución funcional, segura y adaptable a las necesidades del entorno académico.